Decoupling-Based Cross-Layer Connection Removal for Compact Remote Sensing Image Classification

Zhiyuan Deng, Yuqi Han, Wenzheng Wang, and Chenwei Deng, *Senior Member, IEEE*

***Abstract*—Cross-layer connections can couple different layers of neural networks in diverse paths, improving the classification accuracy of remote sensing images. However, some structures are unnecessary for inference and not conducive to be deployed on starboard/airborne platforms. Roughly removing them lengthens the backpropagation path and causes the gradient vanishing problem, degrading the performance of existing end-to-end training-based lightweighting methods. To address this issue, we propose a decoupling-based cross-layer connection removal (DCCR) framework, which decomposes the complete deep network into multiple shallower stages based on the density of connections. By optimizing each stage independently, the backpropagation path is shortened and the gradient vanishing problem is expected to be alleviated. Moreover, to approximate the original network in each stage, we perform concrete reconstruction error suppression on branches, layers, and channels. Combining these strategies, redundant structures are removed while maintaining performance. Extensive experiments on multiple public datasets and hardware platforms demonstrate that this compact architecture achieves substantially improved deployability and inference efficiency at the cost of only marginal accuracy degradation.**

***Index Terms*—Cross-layer connection, decoupling, model compression, multibranch neural network, reconstruction, remote sensing image classification.**

I. Introduction

MULTIBRANCH convolutional neural networks based on cross-layer connection (e.g., ResNet [1], DenseNet [2], CSPNet [3], and ELAN [4]) have shown brilliant performance in real-time identification of optical remote sensing objects on satellite and airborne platforms. Compared with single-path networks (e.g., AlexNet [5] and VGG [6]), cross-layer connections couple inputs and outputs from different layers in diverse paths. This means that feature and gradient information can be shared across deep and shallow networks, which shortens the backpropagation path and alleviates the gradient vanishing problem [7]. Consequently, cross-layer connections enhance the ability of neural networks to learn object features of different categories, morphologies, and dimensions from datasets [4] and are commonly used in the design of backbone networks for remote sensing image classification [8], [9], [10], [11], [12], [13]. Even in Transformer-based neural networks [14], [15], [16], the way interlayer connections are constructed remains a key factor affecting model accuracy and efficiency [17], [18].

Although cross-layer connections improve the classification accuracy, efficiency issues arise when deployed on resource-constrained platforms such as spaceborne and airborne. Take Fig. 1 as an example, cross-layer connections allow shallow networks to be connected to deeper networks with shortcuts, while some extra structures (branches, layers, and channels) are needed. These structures can be preserved at low throughput but introduce severe redundancy for real-time processing of high-resolution remote sensing image, which is illustrated as follows.

1) *Branch Redundancy:* Extra branches refer to processing paths with fewer/no parameters, and are typically utilized to cache shallow network output for deep network invocation. These redundancies will cause some intermediate results not being released untill all branches are processed, bringing about a higher peak memory footprint.
2) *Layer Redundancy:* Extra layers refer to those pointwise convolutions in which the sense field is not expanded and are typically utilized to temporarily adjust channel numbers. These redundancies increase the frequency of data access, resulting in a higher access time overhead.
3) *Channel Redundancy:* Extra channels refer to those convolutional kernels that have less impact on the inference result after removal, and are typically utilized to ensure that the dimensionality is consistent before the feature maps are summed up. These redundancies lead to unnecessary parameters and operations.

In order to improve the inference efficiency, it is necessary to remove these concomitant redundant structures of cross-layer connections at low loss.

The difficulty in removing these structures lies in the fact that neural networks are highly nonlinear; roughly replacing them with simplified structures introduces significant errors. Connections in linear operations can be removed losslessly through reparameterization technique in methods such as RepVGG [19], ACNet [20], and RepViT [18], but not in nonlinear structures such as cross-layer connections. Furthermore, these errors are difficult to be recovered through end-to-end training due to the gradient vanishing problem

Received 18 July 2025; revised 12 September 2025 and 27 October 2025; accepted 20 November 2025. Date of publication 25 November 2025; date of current version 9 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62171040 and in part by the YoungElite Scientists Sponsorship Program by China Association for Science and Technology (CAST). *(Corresponding authors: Yuqi Han; Chenwei Deng.)*

The authors are with the School of Information and Electronics, National Key Laboratory of Science and Technology on Space-Borne Intelligent Information Processing, Beijing Institute of Technology, Beijing 100081, China (e-mail: yatsihan@126.com; cwdeng@bit.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TGRS.2025.3636678, provided by the authors.

Digital Object Identifier 10.1109/TGRS.2025.3636678

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

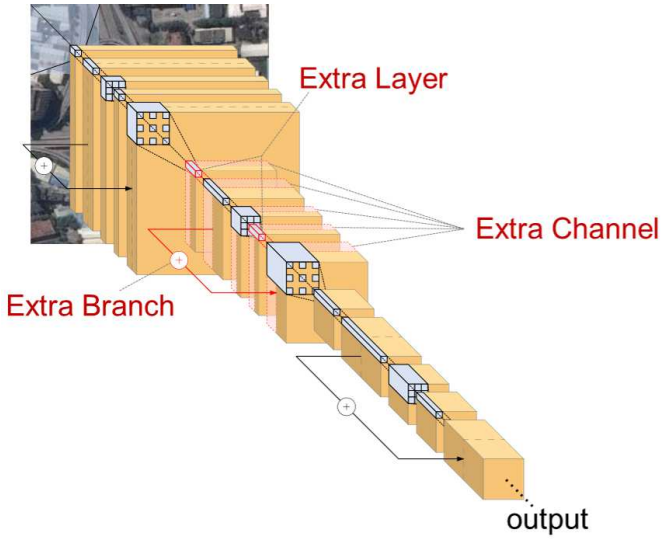


Fig. 1. Schematic of cross-layer connections in a network, where the yellow and blue cubes represent the feature maps and convolutional layers, respectively. The extra branch, layer, and channel indicate the concomitant redundant structures in it.

after the entire network degrading into a deep single-path one. Thus, cross-layer connections need to be preserved in end-to-end training-based lightweighting methods such as ELC [21] and DepthShrinker [22]. Consequently, it is difficult for existing methods to balance redundancy and accuracy when dealing with the above three types of concomitant redundant structures.

In order to address the issues mentioned above, we propose a novel decoupling-based cross-layer connection removal (DCCR) method by decoupling the complete network into multiple stages. Compared with end-to-end training, optimizing on a stage-by-stage basis shortens the backpropagation path, which enables a better approximation of the original network.

Specifically, to ensure the optimization performance of each stage, we carefully design the decoupling criteria and reconstruction approaches. For **extra branch**, we incrementally suppress the secondary branches with/without parameters to mitigate the mutual amplification of errors. For **extra layer**, we construct a single-layer approximation of cascaded pointwise convolution utilizing the feature prior distribution. For **extra channel**, we predict the effect of removing a specific channel on the reconstruction error via Taylor expansion. Combining these strategies above, the gradient vanishing problem caused by removing cross-layer connections is circumvented. This allows us to circumvent the gradient vanishing problem caused by removing cross-layer connections. Extensive experiments on three remote sensing image classification datasets NWPU-RESISC45 [23], UC-Merced Land-Use [24], and AID [25], as well as a fine-grained classification dataset CUB-200-2011 [26] have demonstrated that the backbone networks derived from the proposed DCCR method outperform other backbone networks in metrics such as classification accuracy, memory footprint, and inference speed.

The major contributions of this article are as follows.

1) We propose a decoupling-based lightweighting method to remove cross-layer connections with low loss. By decomposing the optimization of the complete network into stages, we circumvent the gradient vanishing problem in end-to-end training after the network degenerates into a single branch one.
2) To mitigate reconstruction error caused by branch removal, we propose a stage branch merge (SBM) strategy tailored to different scenarios, such as whether branches contain parameters. By constructing an exponential decay coefficient and an L2 regularization term, this strategy approximates a single-branch network from a well-trained multibranch one.
3) To address the redundancy in convolutional layers after branch removal, we propose a linear approximation-based cascaded pointwise convolution fusion method termed SLF, and eliminate the accumulated error during this process.
4) To perform channel pruning on single-branch networks that are unsuitable for end-to-end training, we propose a channel pruning method termed SCP, based on the second-order Taylor expansion of reconstruction error. This approach accomplishes minor channel filtering and accuracy loss recovery without end-to-end training.

II. Related Works

In this section, we briefly describe the principles and limitations of cross-layer connections in remote sensing image classification tasks. Specifically, in Section II-A, we summarize some typical cross-layer connection designs for remote sensing image classification. In Section II-B, we describe existing neural network simplification methods for the three redundancy categories of branch, layer, and channel as well as their limitations.

A. Cross-Layer Connections in Image Classification

Image classification is a basic task for intelligent interpretation of remote sensing images and strongly support downstream tasks such as object detection [27], [28], [29], [30], [31], multisource classification [13], [14], [15], [16], and image segmentation [32], [33]. Early models typically adopt single-branch networks (e.g., AlexNet [5] and VGG [6]) as a backbone to extract features. However, single-branch networks cannot extract complex features by stacking layers due to the gradient vanishing problem; thus, the accuracy metrics are not satisfactory.

To address this problem, researchers enhance the feature extraction capability of the backbone network through two ideas: intermodel feature sharing and interlayer feature sharing. GBRCNet [34], MCNN [35], MSCP [36], RSFJR [37], and SF-CNN [38] adopt the former idea. By constructing connections between models, they synthesized the features of several different models with the expectation that they would complement each other. These methods exponentially increase the amount of parameters and operations and are therefore not advantageous in terms of efficiency.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

In contrast, the latter idea avoids introducing redundant models. For example, DCA [39], FACNN [40], and GBNet [41] improve efficiency by constructing connections between different convolutional layers within the model. On this basis, subsequent studies employ multibranch networks obtained by hand design or architecture search as the backbone network, which greatly reduces the difficulty of feature extraction. Most typical cross-layer connections take two forms, cross-layer summation represented by ResNet [1] and cross-layer splicing, represented by DenseNet [2] and CSPNet [3].

The former forms a shortcut structure by adding the inputs and outputs of cascaded convolutional layers. The gradient information of the deeper layers is able to skip the current module by shortcut and directly affect the shallower layers. The latter directly splices feature maps of different layers, which is able to retain more information, but causes an exponential increase in memory usage. Therefore, it is suggested to use only one cross-layer splice within a single stage in CSPNet, which can improve accuracy while avoiding memory explosion [3]. While these cross-layer connections enhance the accuracy in training, they are unnecessary for inference, presenting an opportunity for optimization.

B. Neural Network Simplification

Roughly removing cross-layer connections degrades the network to a single-path one, which faces the gradient vanishing problem during end-to-end training. Thus, to the best of our knowledge, current approaches typically avoid outright removal of cross-layer connections and instead trim less critical redundancies that minimally affect accuracy in end-to-end training. For better understanding of this issue, we present several typical approaches and their limitations in the following.

1) Branch Merge: In mainstream multibranch neural networks, convolutional layers may have multiple branch outputs that are processed by different convolutional layers and then summed or spliced. Since convolutional layers are usually cascaded with nonlinear activation functions whose processing order cannot be changed, branches containing nonlinear layers cannot be merged with other branches by linear operations.

Therefore, methods such as RepVGG [19] and ACNet [20] try to build branching structures using only branches that do not contain nonlinear layers and merge these branches equivalently through linear operations. These reparameterization methods can losslessly merge branches without nonlinear layers but cannot be applied to branches with nonlinear layers such as residual structures and CSP structures. Therefore, there are obvious limitations for existing methods to deal with complex multibranch networks.

2) Layer Fuse: Point-wise convolutions with a kernel size of 1×1 are often used to extract features of single pixel or adjust the channel number of feature maps in multibranch neural networks. Although a 1×1 convolution is only one-ninth the computation of a 3×3 convolution with the same number of input and output channels, it still significantly increases the data access frequency. For high-speed processing devices that rely on off-chip storage such as GPUs, increased access frequency leads to increased model processing latency.

To solve this problem, methods such as ELC [21] and DepthShrinker [22] achieve the fusing of those convolutional layers with single output and posterior layers by filtering and removing secondary nonlinear layers. These methods can reduce the number of redundant convolutional layers without changing the topology of the model. However, they cannot be directly applied to convolutional layers with multiple outputs, but only maintain the status quo ante or merge the entire module. This means that either too much redundancy will be retained or accuracy will be severely compromised.

3) Channel Prune: For a convolutional layer with known inputs, the number of output channels is equal to the number of convolutional kernels, which directly affects the amount of parameters and computations of current layer and indirectly affects the next one. Therefore, pruning channels is intuitive and effective in reducing the amount of parameters and computations. Its difficulties mainly include two aspects: 1) selection of rational evaluations to distinguish between important/nonimportant channels and 2) synchronized modification of all correlation channel numbers to ensure that the dimensions of the tensor involved in the operation are the same.

For the former problem, there are two main solutions, magnitude-based and sensitivity-based [42]. Magnitude-based methods consider smaller weights to have less impact on the network and are typically used in conjunction with sparsification techniques [43], but drastically change the original distribution of the model parameters. Sensitivity-based methods directly evaluate the effect of the channel on the loss function [44] but are difficult to implement due to the mutual coupling of the parameters. To solve the latter problem, a general method DepGraph [45] was proposed to analyze the dependencies of various network structures (e.g., CNNs, RNNs, GNNs, and Transformers). However, since redundancy varies across feature maps, these methods retain some of the redundant channels or remove significant channels to ensure dimensionality consistency. As a result, redundant channels typically cannot be maximally removed.

III. Proposed Methodology

In this section, we will introduce the details of removing redundant structures of cross-layer connections stage by stage. In Section III-A, we describe the overall framework of the proposed method. In Sections III-B–III-D, specific processing ideas for each of the three types of concomitant redundancies, branch, layer, and channel, will be described.

A. Overall Framework

As shown in Fig. 2, the processing flow of our proposed DCCR method presupposes decomposing the complete network into multiple stages. Subsequently, we perform branch merging, layer fusion, and channel pruning stage by stage to obtain its simplified form. Next, we fine-tune the simplified stage using gradient descent to minimize its MSE loss relative to the original stage, ensuring that its output closely approximates the original output values. Finally, the original stages are substituted with the simplified ones. With this, we accomplish cross-layer connection removal without end-to-end training.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

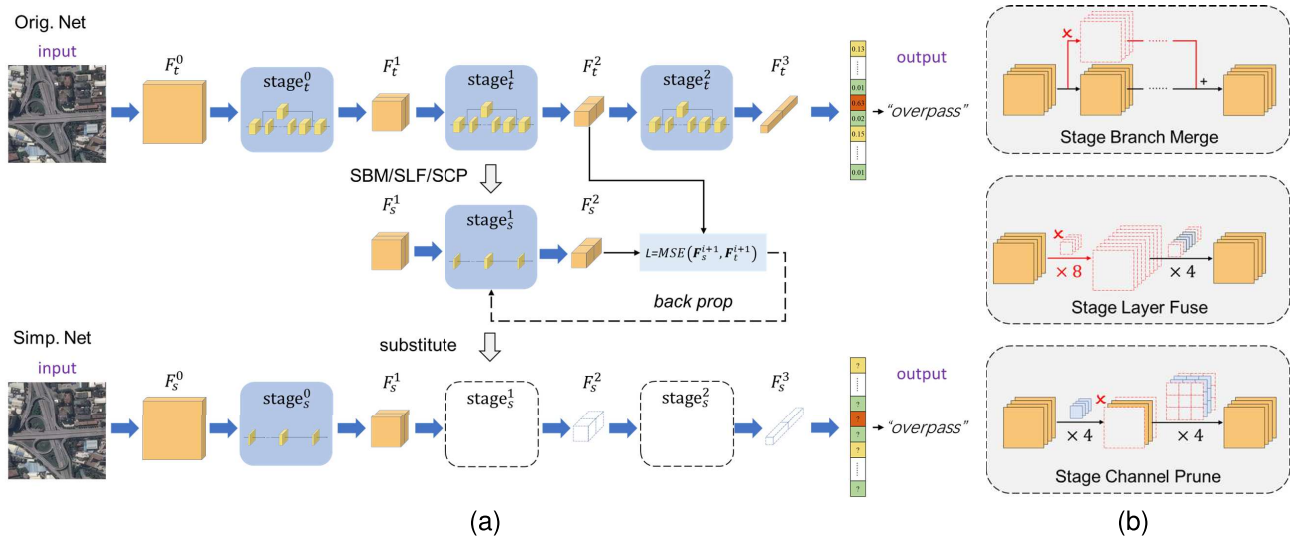


Fig. 2. Schematic of (a) stage-wise decoupled training and (b) three simplification algorithm (SBM, stage layer fuse, and stage channel prune) in DCCR. Darker yellow rectangles represent significant feature map channels and light yellow rectangles represent minor feature map channels. The $k \times k$ light blue rectangle represents a $k \times k$ 2-D weight matrix. The number of stacked matrices represents the input channel number of a single convolutional kernel.

This approach draws inspiration from the greedy algorithm, where in each optimization round, we prioritize removing the structure that is currently easiest to eliminate, repeating this process over multiple rounds until the performance metrics are met. For maintaining accuracy, each stage needs to be relatively independent; otherwise, it is difficult to ensure that the optimum of each stage approximates the optimum of the whole network. Considering that cross-layer connections require consistent feature map dimensions, mainstream networks can typically be divided into multiple shallower stages bounded by downsampling layers. Although there are complex cross-layer connections within each stage, the connection between stages is unique. Thus, we can achieve the simplification of each stage by minimizing the mean square error between the outputs of simplified stage and original one.

Moreover, the inputs and outputs of stages are typically high-dimensional feature maps; improper initialization induces more serious convergence problems compared with end-to-end training. Therefore, we design corresponding reconstruction methods according to the properties of particular structures (branch, channel, and layer), which utilize the original parameters to construct the simplified stages. The specific implementation will be described in Sections III-B–III-D.

B. Stage Branch Merge

Extra branches are the most typical redundant structure for cross-layer connections, and the increase of their amount consumes more memory. Furthermore, the existence of branches increases the complexity of the network topology and is not conducive to the optimization of subsequent layers and channels. Therefore, in this section, we distinguish between the two cases of whether a branch contains parameters or not and merge the dual branches into one.

First, we describe the mathematical model of the SBM problem. As shown in the following equation, this process can be described as replacing the original model with a single branch while minimizing the reconstruction error:

$$\min_{\widetilde{S}} \left\| M(x) - \widetilde{S}(x) \right\|_2 \tag{1}$$

where $M(x)$ denotes a multibranch network and $S(x)$ denotes a single-branch network. For typical multibranch models ResNet and CSPNet, $M(x)$ can be expressed as the sum of a primary branch $S_0(x)$ containing most of the parameters and a secondary branch $S_1(x)$ with few/no parameters, so the formula can be further expressed as follows:

$$\min_{\widetilde{S}} \left\| S_0(x) + S_1(x) - \widetilde{S}(x) \right\|_2 \tag{2}$$

where $S(\boldsymbol{x}) = C_m \circ \cdots \circ C_1 \circ C_0(\boldsymbol{x})$ is a cascade of m convolutional modules C. The layers of primary branches m_0 are typically greater than the layers of secondary branches m_1. Each convolution module typically consists of a convolution operation, a batch normalization layer, and a nonlinear activation function, i.e.,

$$C(\boldsymbol{x}) = f(\mathrm{bn}(W * \boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\gamma}, \boldsymbol{\beta})) \tag{3}$$

$$\mathrm{bn}(x; \mu, \sigma, \gamma, \beta) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \tag{4}$$

where $f(x)$ is a nonlinear activation function (e.g., ReLU, LeakyReLU, and SiLU [46], [47]), $\mathrm{bn}(x; \mu, \sigma, \gamma, \beta)$ is a batch normalization layer, W is the weight of the convolutional layer, and $*$ is the convolution operation.

Due to the underdetermination of the equations and the high dimensionality of the real input x, we can hardly guarantee an analytical solution to this problem. When solving this problem with gradient descent method, the following two types of problems arise.

1) *Convergence:*
According to the 5, the local rate of change of the MSE loss function is related to $y - x$, which does not satisfy global Lipschitz continuity. When measuring

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

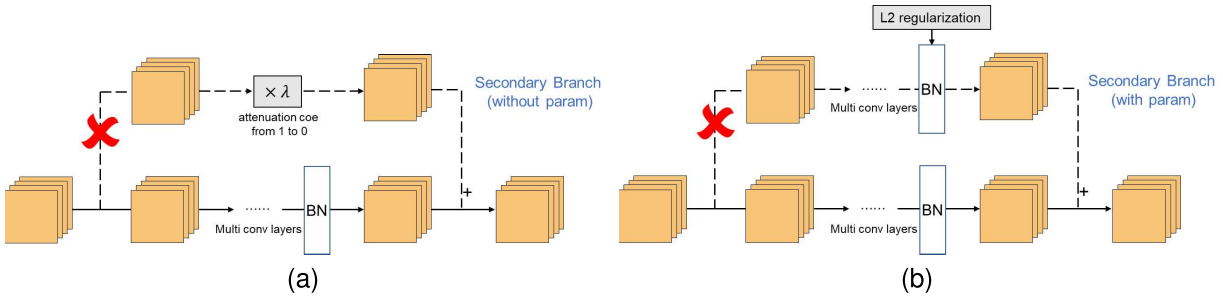


Fig. 3. Schematic of the two-branch suppression strategies in SBM. (a) Suppression of secondary branches in the residual structure (branches without parameters) by introducing a decay function $\lambda(t)$ that decays from 1 to 0 as epochs t increase. (b) Suppression of secondary branches (branches with parameters) in the CSP structure by L2 regularization.

the Euclidean distance between two high-dimensional feature maps, if the initial distance is too large, even a slight change Δy in the input can lead to a sharp change in the loss function [48]

$$\mathrm{MSE}(x, y+\Delta y) = \mathrm{MSE}(x, y) + \Delta y^2 + 2(y-x)\Delta y. \tag{5}$$

This indicates that training becomes highly unstable when the initial error is too large, whereas no such issue occurs when the initial error is sufficiently small. Therefore, before constructing the simplified stage and fine-tuning parameters using gradient descent, we should minimize the initial error as much as possible.

2) *Local Optimum:* Since multilayer neural networks are nonconvex, it is difficult to find the global optimum of $\widetilde{S}$. Therefore, for ensuring that the convergence point is as close to the global optimum as possible, the parameters of the original multibranch network should be utilized to rationally initialize the single-branch network.

Combining these two points, we believe that avoiding excessive initial errors is the key to speed up convergence and avoid local optima. Since the main role of the secondary branch is to provide diverse gradients during training, less prior knowledge is introduced and fewer features are extracted, we can directly initialize $\widetilde{S}(x)$ to the primary branch $S_0(x)$. This reduces the reconstruction error of the model from $\mathrm{Var}(S_0(x)) + \mathrm{Var}(S_1(x)) + \mathrm{Var}(\widetilde{S}(x))$ to $\mathrm{Var}(S_1(x))$.

In practice, due to the nesting of cross-layer connections and mutual amplification of errors, simply retaining the major branches is not enough to remove redundant branches with low loss. A more effective approach is to gradually reduce their impact on the results. Therefore, we design branch removal strategies for two typical multibranch structures: the residual block where the secondary branches do not contain parameters, and the cross-stage partial block where the secondary branches contain parameters, as follows. Meanwhile, these two branches represent two typical branch structures: cross-layer concatenation and cross-layer addition, making them highly representative.

1) Residual Branch Suppression: As shown in Fig. 3(a), a typical residual block can be expressed as follows:

$$M(x) = S_0(x) + x = C_2 \circ C_1 \circ C_0(x) + x. \tag{6}$$

From 6, it can be obtained that the initial error brought by directly removing the secondary branch $S_1(x) = x$ is $\|x\|_2$. To further minimize this error, we progressively remove the residual structure with the following optimization objective:

$$\min_{\widetilde{S}} \left\| (S_0(x) + x) - \left(\widetilde{S}(x) + \lambda(t)\, x\right) \right\|_2 \tag{7}$$

where decay function $\lambda(t) = e^{-\alpha \cdot t} \in [0, 1]$ decreases exponentially as the number of training epoch t increases. Decay coefficient α is a hyperparameter set based on the total rounds of training to control the decay speed of $\lambda(t)$. By gradually decreasing $\lambda(t)$, we progressively reduce the impact of cross-layer connections to ensure that the absolute error $E_a(t)$ introduced by the tth round of optimization satisfies $E_a(t) \le |(\lambda(t-1) - \lambda(t))x|^2 < |x|^2$.

2) CSP Branch Suppression: As shown in Fig. 3, the structure of cross-stage partial block can be approximated as follows:

$$\begin{aligned} y &= S_0(x) + S_1(x) \\ &= C_{0m} \circ \cdots \circ C_{01} \circ C_{00}(x) + C_{11} \circ C_{10}(x). \end{aligned} \tag{8}$$

Based on the properties of BN layer, we can assume that the output distribution of $S_1(x)$ approximates $N(\|\beta_{11}\|_1, \|\gamma_{11}\|_2)$. where β_{11} can be directly summed with the main branch, so the initial error $E = \|\gamma_{11}\|_2$. Therefore, reducing the effect of branches with parameters can be achieved by decreasing $\|\gamma_{11}\|_2$.

This problem can be equated to a regularization problem in the following equation, an additional regularization term is introduced to γ:

$$\begin{aligned} \widetilde{J} &= J + \frac{s}{2}\|\gamma\|_2 \\ \frac{\partial \widetilde{J}}{\partial \gamma} &= \frac{\partial J}{\partial \gamma} - s \cdot \gamma \\ \gamma &\leftarrow (1 - \varepsilon s)\gamma - \varepsilon \nabla_\gamma J \end{aligned} \tag{9}$$

where J is the reconstruction loss, γ is the scaling factor of the BN layer, ε is the learning rate, and s is the sparsity coefficient. By linearly scaling γ before updating it with gradient descent, γ eventually tends to zero. With this approach, we can gradually suppress the secondary branches of the CSP structure during training.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

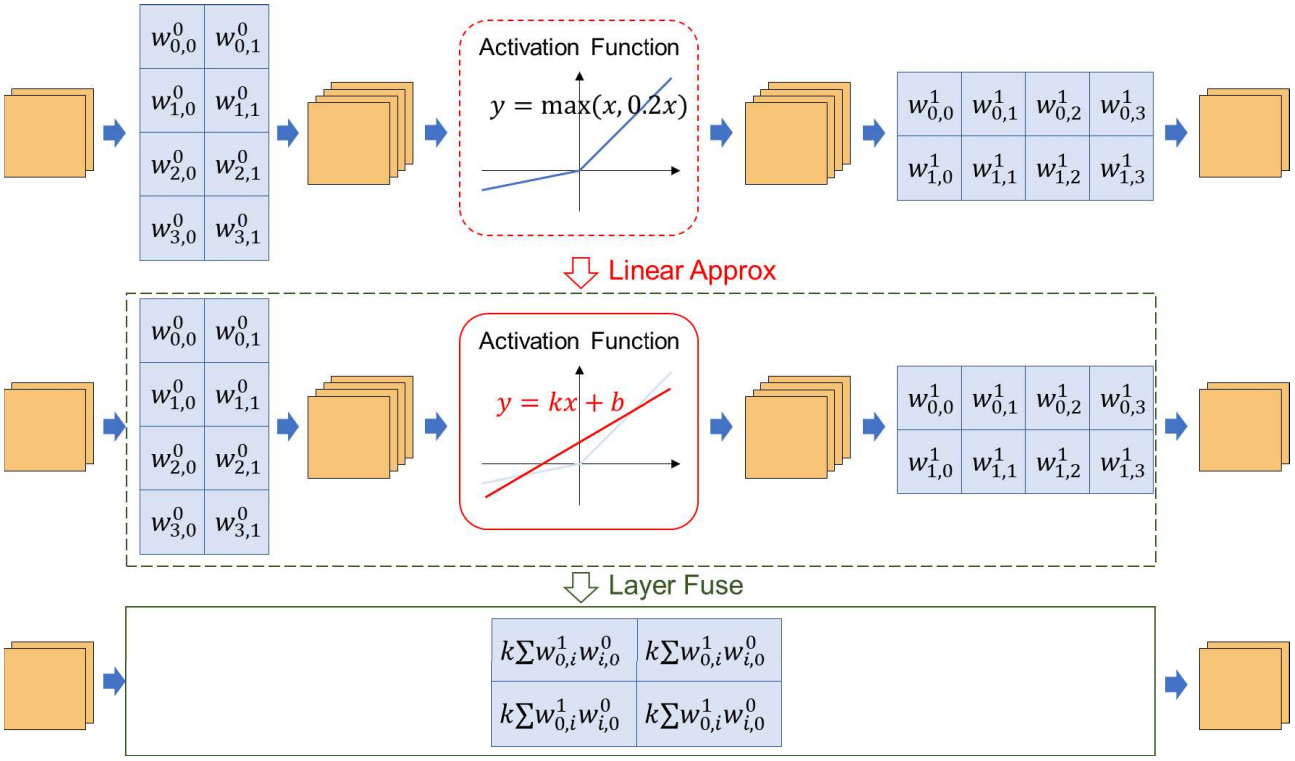


Fig. 4. Schematic of the stage layer fuse processing flow. First, the nonlinear layers are removed by linear approximation. Then, the two cascaded 1 × 1 convolutions are merged by matrix multiplication. Finally, we fine-tune the parameters.

Combining these two points, we can obtain the objective function for optimization in terms of stages, as shown in the following equation:

$$\min_{\text{stage}_s^i} J = \min_{\text{stage}_s^i} \left\| F_t^{i+1} - \text{stage}_s^i \left(F_s^i; \lambda(t) \right) \right\|_2 + \frac{s}{2} \left\| \boldsymbol{\gamma}_{\text{csp}} \right\|_2 \quad (10)$$

where

$$\begin{aligned} F_t^0 &= F_s^0 = X \\ F_t^i &= \text{stage}_t^i \left(F_t^{i-1} \right) \\ F_s^i &= \text{stage}_s^i \left(F_s^{i-1} \right) \end{aligned} \quad (11)$$

where stage_s^i is the ith neural network stage model that introduces the decay function $\lambda(t)$ and $\boldsymbol{\gamma}_{\text{csp}}$ is the scaling factor of the BN layer in the CSP structure.

C. Stage Layer Fuse

In this section, we focus on optimizing redundant layers in cross-layer connection design, where point-wise convolution with a kernel size of 1 × 1 is one of them. Point-wise convolution is unable to expand the receptive field, which is typically utilized to adjust the channel number in cross-layer connection design so as to satisfy the constraints on feature map dimensions. These constraints are eliminated after SBM. Considering that multiple cascaded point-wise convolutions have the same receptive field as a single one, it is necessary to fuse them and reduce the frequency of memory access. Thus, we can define the problem of low-loss fusion as follows:

$$\min_{C} \left\| C_1^{n \times m \times 1 \times 1} \circ C_0^{m \times n \times 1 \times 1}(\boldsymbol{x}) - C^{n \times n \times 1 \times 1}(\boldsymbol{x}) \right\|_2 \quad (12)$$

where $C^{\text{out} \times \text{in} \times 1 \times 1}(\boldsymbol{x}) = f(\text{bn}(W\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\gamma}, \boldsymbol{\beta}))$ denotes a 1 × 1 convolution with the number of input channels in and the number of output channels out. $\boldsymbol{x}$ is an in-dimensional vector consisting of all channel values of a single pixel in input feature map, W is an out × in-sized weight matrix, and $\boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\gamma}, \boldsymbol{\beta}$ are out-dimensional parameters of BN layer. By performing bn fusion referring to (14), we can formulate it in the form of (13)

$$\min_{\widetilde{W}, \widetilde{b}} \left\| W_1 f\left(W_0' \boldsymbol{x} + \boldsymbol{b_0} \right) - \left(\widetilde{W} \boldsymbol{x} + \widetilde{\boldsymbol{b}} \right) \right\|_2 \quad (13)$$

$$W_{i,:}' = \frac{\gamma_i}{\sigma_i} W_{i,:}, b_i' = -\frac{\mu_i \gamma_i}{\sigma_i} + \beta_i \quad (14)$$

where subscript i denotes the element of row i in the corresponding matrix and vector.

The existence of nonlinear activation function f makes this problem difficult to solve analytically, relying on data-driven methods to approximate the solution. Following the conclusions in Section III-B, reasonable initialization of $\widetilde{W}$ can reduce the initial error and speed up convergence. Thus, we can design a linear function to approximate the original nonlinear layer and then fuse the neighboring layers by matrix multiplication.

This processing flow is demonstrated in Fig. 4. We compute the optimal slope k and intercept b using the least-squares method based on the input $u = W_0' \boldsymbol{x} + \boldsymbol{b_0}$ of the nonlinear layer

$$\min_{k,b} \| f(\boldsymbol{u}) - (k\boldsymbol{u} + b) \|_2. \quad (15)$$

Subsequently, we initialize $\widetilde{W}$ and $\widetilde{\boldsymbol{b}}$ to the form in the following equation:

$$\begin{aligned} \widetilde{W} &= k W_1 W_0' \\ \widetilde{\boldsymbol{b}} &= W_1 (k \boldsymbol{b_0} + b). \end{aligned} \quad (16)$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

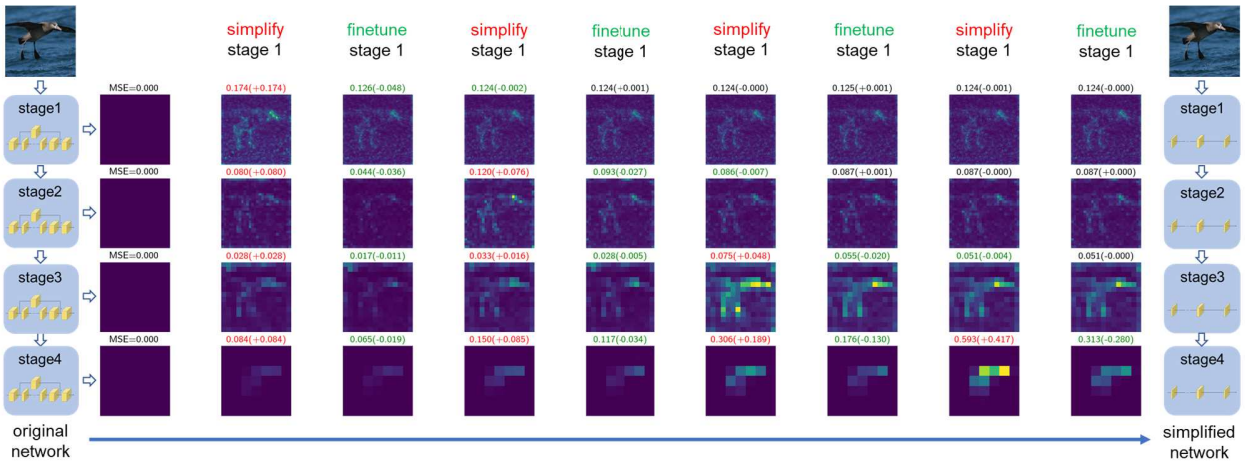


Fig. 5. Error heatmaps of four-stage outputs during stage-wise simplification and fine-tuning in baseline model. From (left to right): after simplifying Stage 1, after fine-tuning Stage 1, after simplifying Stage 2, and so forth. Brighter colors in the heatmaps indicate larger errors.

Doubtless, the linear approximation inevitably introduces errors that are small for most typical values and large for a few outliers. These errors can gradually accumulate and amplify each other during model inference. Additionally, greedily optimizing the optimal result for each individual convolutional layer does not necessarily lead to the optimal outcome for the entire stage or network. Therefore, after simplifying a particular stage, we need to perform multiple rounds of fine-tuning based on the following equation:

$$\min_{\text{stage}_s^i} J = \min_{\text{stage}_s^i} \left\| F_t^{i+1} - \text{stage}_s^i\left(F_s^i\right) \right\|_2. \tag{17}$$

In contrast to (10), we use the actual inputs of the stage rather than the original inputs as excitations in order to correct for cumulative errors. This operation allows the optimization of each stage to be closer to the global optimum, which effectively reduces the impact of redundant structure removal on the final accuracy of the model.

Fig. 5 illustrates the impact of simplification and fine-tuning on stage-wise errors. When optimizing the ith stage, the errors of both the current stage and subsequent stages increase significantly. However, after multiple rounds of fine-tuning, the errors of this stage can be reduced to a relatively low level. Since stage-by-stage simplification proceeds from shallow to deep stages, the accumulated errors are gradually mitigated during fine-tuning, thereby minimizing the final stage error as much as possible.

D. Stage Channel Prune

The number of convolutional layer channels directly affects the number of parameters and operations of the model. At the outset of designing a multibranch network, they are typically specified manually. However, the practical importance of different channels varies, and removing some of them has a small effect on the accuracy but significantly improves the speed. Therefore, in this section, we focus on solving the identification and low-loss removal of redundant channels.

Generally, channel pruning requires both filtering out redundant channels and retraining to recover accuracy. Cross-layer connections in a multibranch network impose the constraint that specific feature maps must have the same dimensionality, so some redundant channels need to be preserved. After SBM and SLF, only a single branch exists in the model and the number of channels is no longer constrained. However, the cost of flexible tuning of the channel number for single-branch networks is that end-to-end training becomes difficult. Pruning methods based on end-to-end training are no longer applicable in this case. Therefore, we need to follow the training strategy of local reconstruction in Sections III-B and III-C. The reconstruction loss of the ith stage of the neural network can be defined as follows:

$$L\left(F_t^{i+1}, F_s^i; \theta\right) = \left\| F_t^{i+1} - S\left(F_s^i; \theta\right) \right\|_2. \tag{18}$$

Similar to the definition in (11), F_s^i is the actual input of the ith stage, F_t^{i+1} is the output feature map of the original ith stage, $S(F;\theta)$ is the pruned stage we need to construct, and θ is the parameters in this model.

On this basis, we can construct the mathematical definition of the pruning task. Specifically, we use $\mathcal{K}$ to represent the set of all channels in the stage. Filter pruning is to choose a subset of filters $k \subset \mathcal{K}$ and remove their parameters θ_k^- from the stage. We note the left parameters as θ_k^+; therefore, we have $\theta_k^+ \cup \theta_k^- = \theta$. To minimize the loss increase, we need to carefully choose the prune set k^* by solving the following optimization problem:

$$k^* = \underset{k}{\text{argmin}} |L\left(F_t^{i+1}, F_s^i; \theta_k^+\right)|, \text{s.t.} \|k\|_0 \geq \|\mathcal{K}\|_0 \cdot R_{\text{pr}} \tag{19}$$

where $\|k\|_0$ and $\|\mathcal{K}\|_0$ are the elements number of k and $\mathcal{K}$, respectively, and R_{pr} is the prune ratio selected on demand. A simple way to solve this problem is to try out all possibilities of k and choose the best one that has the least effect on the loss. However, this method requires iterating through $C_{\|\mathcal{K}\|_0}^{\|k\|_0} = (\|\mathcal{K}\|_0!/\|k\|_0!(\|\mathcal{K}\|_0 - \|k\|_0)!)$ possible cases, which is infeasible for deep models that has tens of thousands of filters.

To make this problem tractable, we need to assume a certain degree of independence between the channels, so as to independently evaluate the impact of each channel on the model accuracy. According to the mathematical definition of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

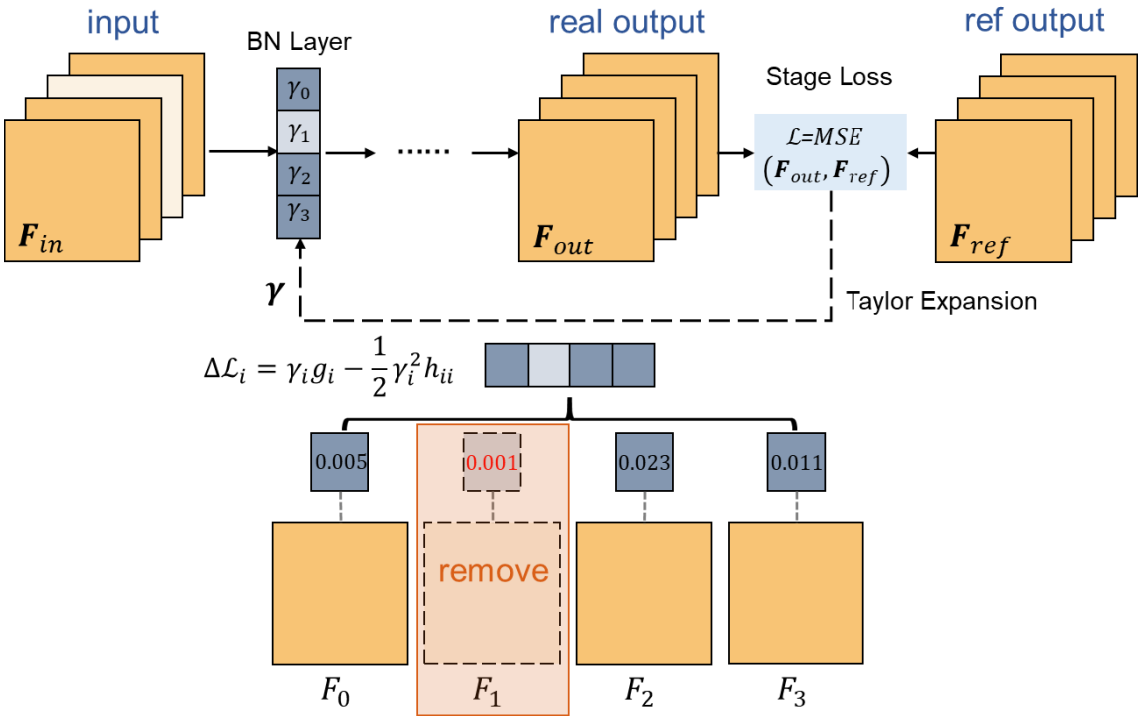


Fig. 6. Schematic of the stage channel prune processing flow. For the scaling coefficient γ in the batch normalization layer corresponding to each channel, we calculate its second-order Taylor expansion to estimate its importance.

the BN layer in (4), we can get that when the ith element $\gamma_i = 0$ in $\boldsymbol{\gamma}$, the output of the corresponding ith channel is the constant β_i independent of the input x. Therefore, removing specific channels in the convolutional kernel can be realized by setting the corresponding γ_i to zero. On this basis, we approximate the importance of each channel by solving the effect of γ_i setting to zero on the reconstruction error $\mathcal{L}(x)$ via Taylor expansion

$$\mathcal{L}(x) = \mathcal{L}(\gamma_i) + \mathcal{L}'(\gamma_i)(x - \gamma_i) + \frac{1}{2}\mathcal{L}''(\gamma_i)(x - \gamma_i)^2 + R_2(\gamma_i). \tag{20}$$

Thus, the error of setting γ_i to zero can be expressed as follows:

$$\begin{aligned}\Delta\mathcal{L}(\gamma_i) &= \mathcal{L}(\gamma_i) - \mathcal{L}(0) \\ &= \gamma_i\mathcal{L}'(\gamma_i) - \frac{1}{2}\gamma_i^2\mathcal{L}''(\gamma_i) - R_2(\gamma_i) \\ &\approx \gamma_i\mathcal{L}'(\gamma_i) - \frac{1}{2}\gamma_i^2\mathcal{L}''(\gamma_i)\end{aligned} \tag{21}$$

where the gradient $\mathcal{L}'(\gamma_i)$ in the first-order term can be obtained directly in the backpropagation. The second-order term involves second-order partial derivatives $\mathcal{L}''(\gamma_i)$, which require computing the Hessian matrix (essentially taking the derivative of the gradient). This operation has a computational complexity of $O(N^2)$ (N is the number of parameters), making it impractical in applications. Therefore, we ignore the interactions between parameters and approximate using only the diagonal elements of the Hessian. As for higher order residual terms in $R_2(\gamma_i)$, their computation is prohibitively expensive and offers limited benefits (the minor loss in accuracy can be recovered by increasing the number of fine-tuning iterations). Thus, they are not considered in this work.

The entire process of identification and removal is demonstrated in Fig. 6. After ranking the importance of channels, the secondary ones are removed according to the preset pruning ratio. Then, we fine-tune the parameters based on the method of local training in (17). In particular, since the computation of the reconstruction error requires the feature map to be dimensionally consistent, the output channels of the last layer are not pruned.

IV. Experiments

In this section, extensive experiments are performed to verify the effectiveness of the proposed DCCR method in improving efficiency while maintaining accuracy. In Section IV-A, we briefly introduce the datasets utilized. In Section IV-B, we describe the selection of the baseline method and the optimizer settings. In Section IV-C, we conduct ablation experiments on the effects of the three components of DCCR (SBM, SLF, and SCP) on the model efficiency metrics. In Section IV-D, we further investigate the impact of decoupling strategy and these three components on accuracy. In Sections IV-E and IV-F, a systematic comparison is presented between DCCR-generated models and mainstream networks, evaluated on both remote sensing datasets (NWPU-RESISC45, UC-Merced Land-Use, and AID) and fine-grained dataset (CUB-200-2011). In Section IV-G, we futher investigate the generalizability of the DCCR method extended to the object detection tasks (DOTAv1.0). In Section IV-H, we further validate the benchmarking of the method on edge devices that are critical for satellite/airborne deployments.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

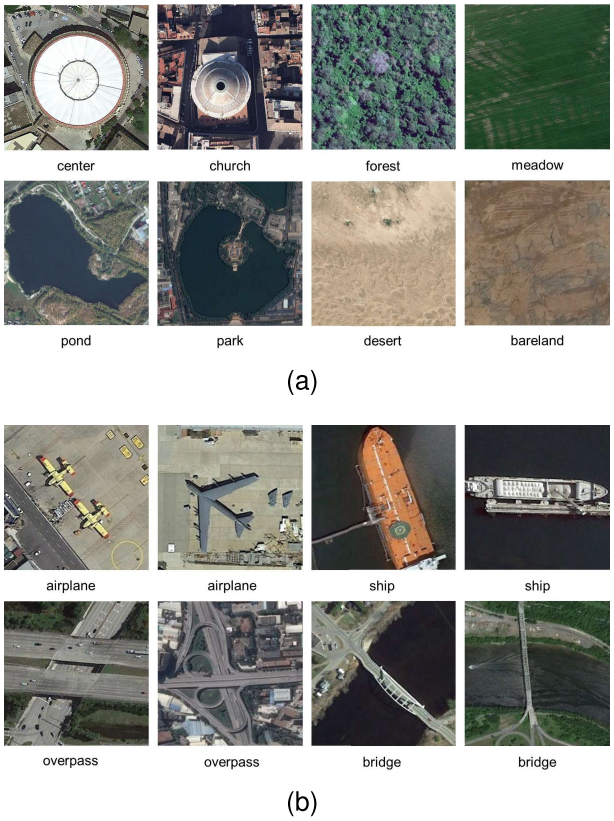


Fig. 7. Examples of (a) dissimilar targets with similar morphology in the AID dataset and (b) similar targets with dissimilar morphology in the NWPU-RESISC45 dataset.

A. Datasets

We conducted experiments mainly on three widely used remote sensing image classification datasets (NWPU-RESISC45, UC-Merced Land-Use, and AID) to validate the performance of the proposed framework. Some confusing examples of these datasets are shown in Fig. 7.

NWPU-RESISC45 [23] dataset is a remote sensing image scene categorization dataset created by Northwestern Polytechnical University (NWPU), which contains a total of 31 500 images with a pixel size of 256 × 256, covering 45 scene categories, of which there are 700 images in each category. Except for a few scene images with low spatial resolution, such as islands, lakes, and mountains, most of the images have a spatial resolution of 30–0.2 m.

AID [25] is a large aerial image dataset made up of satellite images acquired from Google Earth imagery of different countries and regions, which consists of 10 000 single-label images categorized into 30 classes. The number of samples per class varies from 220 to 420. Each image is 600 × 600 pixels in RGB space with spatial resolution varying from about 8 to about 0.5 m.

UC-Merced Land-Use dataset [24] is a benchmark dataset for remote sensing and land-use classification research, consisting of 2100 aerial images across 21 distinct categories. Each category contains 100 images (256 × 256 pixels) with a ground resolution of 0.3 m, sourced from the USGS National Map Urban Area Imagery database in the United States.

TABLE I
OPTIMIZER SETTINGS FOR THE BASELINE MODEL

Method	SGD
learning rate	0.01
momentum	0.9
weight decay	0.0005

Furthermore, we evaluate the method's generalizability on fine-grained classification (CUB-200-2011) and remote sensing object detection (DOTA-v1.0) tasks.

CUB-200-2011 [26] is a fine-grained classification dataset of 200 bird species with 11 788 images, each labeled with the frame, keypoints, and attribute categories of the object, which is commonly used in small-sample fine-grained image classification tasks [49], [50], [51].

DOTA [52] is the largest public dataset for remote sensing object detection. DOTA-v1.0 contains 15 categories, 2806 images, and 188 282 annotation examples. Since the size of images can be up to 20 000 × 20 000 pixels, we cut the original image into 1024 × 1024 pieces with 100 overlapping pixels.

B. Implementation Details

1) Baseline Model: For the baseline model selection, we adopted CSPResNet50 [3] with four stages, which is extensively used as the backbone network for classification and detection models. In addition to its ability to achieve objectively high accuracy across various scenarios, CSPResNet offers the following advantages.

1) CSPResNet incorporates both the representative cross-layer addition operation from ResNet and the cross-layer concatenation operation from DenseNet, which makes it particularly suitable for demonstrating the adaptability of our DCCR method in handling different types of cross-layer connections.
2) CSPResNet primarily consists of hardware-friendly 3 × 3 and 1 × 1 convolutions. The 3 × 3 convolutions can be significantly accelerated on hardware platforms through the Winograd method [53], while the 1 × 1 convolutions can be efficiently merged via our proposed SLF method, achieving notable compression and speedup.
3) After DCCR optimization, CSPResNet is transformed into a single-path network composed solely of standard convolutions and activation functions in series. This provides significant deployment advantages, as it eliminates the need for specialized hardware/software implementations of complex operators (e.g., channel attention, tensor splitting/concatenation, and layer normalization).

2) Optimizer Setting: The experimental setup mainly consists of both optimizer configurations for baseline model training and stage-wise finetuning in the DCCR method. For the training of baseline model, we referred to the original hyperparameter settings of CSPNet [3] and utilized the SGD optimizer (see Table I for detailed configurations), beginning with a five-epoch warmup period before switching to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

TABLE II
OPTIMIZER SETTINGS FOR THE DCCR METHOD

Method	AdamW
learning rate	0.001
β_1	0.9
β_2	0.999
weight decay	0.0001

cosine annealing for learning rate scheduling. For the stage-wise finetuning during the DCCR method, we utilized the AdamW optimizer (see Table II for detailed configurations). We referred to the common settings of betas and weight decay [54], [55], and searched for a suitable learning rate on the CUB-200-2011 dataset, which was subsequently applied to other datasets. Specifically, we started with relatively large learning rates (e.g., 0.1) and gradually reduced them by an order of magnitude each time, selecting the setting that achieved the fastest convergence. All experiments were conducted with mmpretrain and pytorch frameworks.

C. Ablation Studies on Efficiency

In order to evaluate the impact of removing the three types of redundancy on efficiency, we conducted a set of ablation experiments to evaluate its impact on storage and speed. In these experiments, we sequentially performed SBM and SLF and then performed SCP with different pruning ratios. Synchronously, we counted three types of storage metrics, including parameter amount, activation amount, and maximum number of simultaneously cached feature maps, as well as four speed metrics, including theoretical operations FLOPS, convolutional layer number, inference latency, and FPS on an RTX3060 Laptop GPU.

The effects of each step on these metrics were shown in Table III. Among them, SBM had less impact on the amount of parameters and operations, but significantly reduced the peak memory footprint of the model (by about 75%). At the same time, SBM transformed the model into a single-branch one, which provided a prerequisite for subsequent simplification. SLF significantly reduced the amount of parameters and operations, as well as the number of convolutional layers. This optimization effectively reduced the frequency of accessing data, and the improvement in actual inference speed on the GPU was significant. SCP further compressed the amount of parameters and operations of the model, and the compression ratio increased with the pruning ratio, which can be flexibly adjusted according to the actual speed index in the application.

In addition, these results indicated two points.

1) The memory footprint was affected by the number of branches and the size of feature maps, so reducing the number of branches can effectively reduce the memory footprint.
2) The inference speed was affected by the total computation and the memory access frequency, so reducing the number of layers and channels both reduces the processing latency.

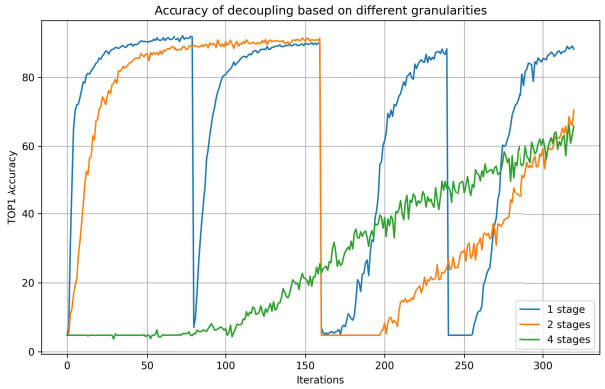


Fig. 8. Trend of model accuracy with the number of iterations during the fine-tuning process on the UC-Merced Land-Use dataset, decoupled in one, two, and four stages.

Therefore, ideally, compression and acceleration of neural networks need to balance these four metrics.

D. Ablation Studies on Accuracy

1) Evaluation on Decouple Granularity: To validate the effectiveness of the proposed decoupling method, we conducted multiple sets of experiments with different decoupling granularities. Since the baseline model can be divided into a maximum of four stages, we performed decoupling at one-, two-, and four-stage units, completing a total of 320 iterations on the UCM dataset.

The initial model accuracy was 94.05%. After decoupling at the first-stage unit (applying SBM, SLF, and fine-tuning), the final accuracy reached 88.15%. When decoupling at two-stage units, the accuracy dropped to 70.42%, and fine-tuning all four stages as a single unit further reduced accuracy to 65.71%. Fig. 8 visualizes the trend of model accuracy during fine-tuning.

The results demonstrated that while increasing decoupling granularity introduced more instability during training, fine-tuning shallow layers independently significantly improved final accuracy. Additionally, since optimizing the ith stage required the outputs of the $(i-1)$th stage of the simplified network and the ith stage of the original one, increasing the decoupling granularity increased the computational load of forward propagation (in an n-stage model, the ith stage requires $n-i+1$ times of forward propagation), while the computational load of back propagation remains largely unchanged. To address this issue, feature maps can be cached on disk.

2) Evaluation on Stage Branch Merge: To evaluate the impact of each component in SBM, we conducted a series of ablation experiments on the AID and CUB datasets. Specifically, we trained the last stage of CSPResNet50 for ten epochs according to (10) (with s set to 0 and 0.01, and α set to 0 and 10), followed by ten epochs of fine-tuning based on (17), obtaining the results shown in Table IV.

The experimental results demonstrate that without using either branch suppression strategy, the accuracy drops by up to 2.71% (CUB) and 2.02% (AID). Suppressing the CSP

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ABLATION STUDY ON COMPONENTS CONTRIBUTIONS FOR EFFICIENCY

Method				Storage Metrics			Speed Metrics			
Branch Merge	Layer Fuse	Channel Prune	Prune ratio	Params (M)	Activations (M)	Cached Feature Maps	FLOPs (G)	Conv Layer Number	Latency (ms)	FPS (Hz)
✗	✗	✗	-	20.796	8.806	4×	3.475	55	6.920	144.51
✓	✗	✗	-	18.703	8.053	1×	3.166	55	4.688	213.30
✓	✓	✗	-	13.159	4.140	1×	2.246	34	4.284	233.43
✓	✓	✓	0.125	12.016	3.989	1×	2.037	34	4.138	241.66
✓	✓	✓	0.25	10.874	3.838	1×	1.828	34	3.796	263.44
✓	✓	✓	0.5	8.589	3.537	1×	1.410	34	2.739	365.10

TABLE IV
ABLATION STUDY OF STAGE BRANCH MERGE

Res Sup	CSP Sup	CUB		AID	
		Init Acc(%)	FT Acc(%)	Init Acc(%)	FT Acc(%)
✗	✗	82.48	79.77(-2.71)	94.93	92.90(-2.02)
✗	✓		80.26(-2.22)		93.08(-1.85)
✓	✗		80.41(-2.07)		93.28(-1.65)
✓	✓		80.93(-1.55)		93.60(-1.33)

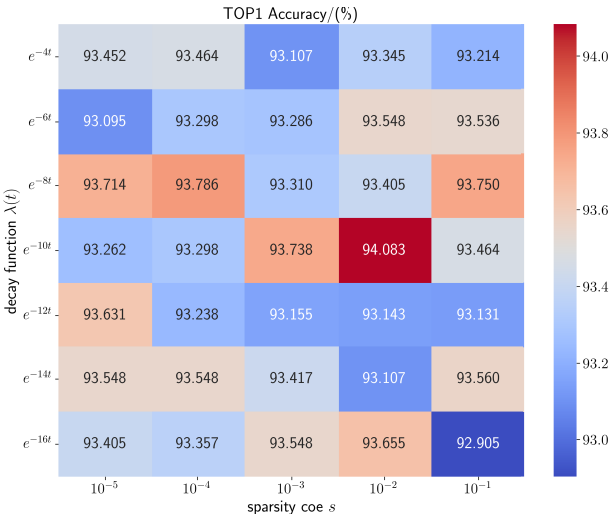


Fig. 9. Grid search results for the sparsity coefficient s and decay function $\lambda(t)$ on the UCM dataset.

structure reduces the accuracy loss to 2.22% (CUB) and 1.85% (AID) while suppressing the residual structure decreases the loss to 2.07% (CUB) and 1.65% (AID). The best performance is achieved when both strategies are applied simultaneously, reducing the loss to 1.55% (CUB) and 1.33% (AID).

The purpose of parameters s and α is to avoid abruptly introducing excessive errors that could disrupt the convergence of the MSE loss function. Therefore, we primarily designed them based on the total training epochs and learning rate and conducted a parameter sensitivity analysis via grid search on the UCM dataset. As shown in Fig. 9, the results indicate that s performs well when set between 0.001 and 0.01, while α yields optimal results in the range of 8–10.

3) Evaluation on Stage Layer Fuse: To evaluate the effect of linear approximation in SLF on the loss of accuracy of fusing redundant layers, we conducted a second set of ablation

TABLE V
ABLATION STUDY OF STAGE LAYER FUSE

Linear Approx	CUB		NWPU	
	Init Acc(%)	FT Acc(%)	Init Acc(%)	FT Acc(%)
✗	82.48	82.08(-0.40)	92.17	91.42(-0.75)
✓		82.17(-0.31)		91.58(-0.59)

TABLE VI
ABLATION STUDY OF STAGE CHANNEL PRUNE

Pruning Metric	Init Acc(%)	Acc(%) under Pruning Ratio		
		0.125	0.250	0.500
x	82.48	80.76(-1.72)	78.08(-4.40)	66.47(-16.01)
g		80.36(-2.12)	79.74(-2.74)	76.22(-6.26)
xg		80.70(-1.78)	80.10(-2.38)	76.67(-5.81)
$xg - \frac{1}{2}x^2h$		81.15(-1.33)	80.17(-2.31)	73.99(-8.49)

experiments on the CUB and NWPU datasets. Specifically, for the final stage of CSPResNet50, we first apply SBM to obtain a single-branch structure, then merge adjacent 1×1 convolutions within it via SLF, and fine-tune for 25 epochs according to (2). The merging is performed using both linear approximation and random initialization. The results in Table V demonstrated that random initialization of the fused convolutional layers brought about a 0.40% accuracy loss on CUB and 0.75% on NWPU, which can be reduced to 0.31% on CUB and 0.59% on NWPU by the linear approximation method. These results demonstrated that the linear approximation method can effectively avoid the error caused by redundant layer fusing.

4) Evaluation on Stage Channel Prune: To evaluate the impact of different parameters in the Taylor expansion on channel importance, we conducted a comparative experiment on the CUB dataset. Specifically, we used the following metrics as criteria for channel importance: the magnitude of BN layer gamma coefficients, gradients, first-order Taylor expansion, and second-order Taylor expansion. We then pruned the model at different ratios across its four stages (after SBM and SLF processing), with each stage undergoing ten epochs of fine-tuning after pruning.

The experimental results (Table VI) demonstrate that when the pruning ratio is low, the second-order Taylor expansion performs best. However, as the pruning ratio increases, the first-order Taylor expansion alone achieves competitive results.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE EVALUATION ON REMOTE SENSING CLASSIFICATION DATASETS

Neural Network	params (M)	flops (G)	activations (M)	Accuracy (%) AID	NWPU	UCM	Latency (ms)
CSPResNet50(baseline) [3]	20.796	3.475	8.806	93.97(±0.34)	91.70(±0.37)	94.83(±0.51)	6.92(±1.32)
ResNet50 [1]	23.918	4.109	11.114	90.43(±1.30)	86.76(±1.16)	89.83(±0.39)	6.66(±1.27)
DenseNet121 [56]	7.159	2.865	6.899	91.45(±0.57)	89.42(±0.35)	90.11(±0.65)	15.73(±1.70)
RepVGG-A2 [19]	24.373	5.116	3.130	90.99(±0.83)	88.42(±0.75)	88.18(±3.37)	4.46(±0.83)
EfficientNetV2-b0 [57]	6.115	0.728	4.772	93.54(±0.28)	90.52(±0.23)	**94.37**(±0.41)	10.05(±1.42)
EfficientNet-b0 [58]	4.264	0.400	6.747	93.36(±0.44)	90.23(±0.64)	94.12(±0.81)	9.63(±1.46)
MobileNetV3-large [59]	4.458	0.224	4.405	92.30(±0.67)	90.09(±0.76)	91.10(±0.58)	6.24(±1.15)
MobileNetV3-small [59]	1.922	0.058	1.416	88.68(±1.15)	83.23(±0.65)	87.68(±1.57)	5.51(±1.15)
ShuffleNetV2 [60]	1.459	0.148	1.950	90.90(±0.58)	87.94(±0.50)	91.06(±1.80)	5.94(±1.14)
Rev-ViT-small [61]	21.819	4.583	11.873	88.66(±0.67)	82.70(±1.29)	87.77(±2.37)	10.18(±1.46)
MViTv2-tiny [62]	23.558	4.704	21.155	85.76(±0.53)	79.27(±4.48)	84.60(±1.54)	12.90(±1.92)
TinyViT-21m [63]	20.737	4.275	20.060	79.04(±0.87)	79.21(±0.66)	82.32(±1.77)	7.87(±1.51)
RepViT-M1.1 [18]	7.790	1.357	9.423	89.03(±0.60)	88.58(±1.32)	87.35(±1.30)	9.13(±1.25)
RMT-T [64]	14.365	2.650	22.480	93.07(±0.79)	91.11(±0.70)	93.36(±0.67)	17.86(±2.13)
TransNeXt-Micro [65]	12.821	2.663	43.263	90.37(±0.87)	88.86(±1.37)	93.11(±0.97)	35.05(±4.54)
DCCR($R_{pr}=0.000$)	13.159	2.246	4.140	**93.57**(±0.28)	**91.56**(±0.36)	94.27(±0.69)	4.28(±1.60)
DCCR($R_{pr}=0.125$)	12.016	2.037	3.989	93.51(±0.36)	91.51(±0.27)	94.08(±0.70)	4.14(±0.77)
DCCR($R_{pr}=0.250$)	10.874	1.828	3.838	93.28(±0.29)	91.47(±0.36)	93.85(±0.55)	3.80(±0.86)
DCCR($R_{pr}=0.500$)	8.589	1.410	3.537	92.29(±0.37)	90.50(±0.41)	91.88(±1.05)	**2.74**(±0.68)

TABLE VIII
PERFORMANCE EVALUATION ON REMOTE SENSING CLASSIFICATION DATASETS WITH DIFFERENT TRAINING RATIOS

Neural Network	AID 20% training	AID 50% training	NWPU 10% training	NWPU 20% training	UCM 50% training	UCM 80% training
CSPResNet50 [3]	93.98(±0.14)	96.47(±0.27)	91.15(±0.19)	93.36(±0.17)	97.71(±0.47)	99.19(±0.40)
DCCR($R_{pr}=0.000$)	93.60(±0.24)	96.41(±0.25)	90.82(±0.30)	93.07(±0.12)	97.68(±0.45)	98.90(±0.36)
DCCR($R_{pr}=0.125$)	93.36(±0.15)	96.10(±0.25)	90.62(±0.47)	92.83(±0.12)	97.49(±0.34)	98.71(±0.40)
DCCR($R_{pr}=0.250$)	93.28(±0.29)	95.93(±0.31)	90.45(±0.34)	92.73(±0.22)	97.26(±0.55)	98.67(±0.36)
DCCR($R_{pr}=0.500$)	92.08(±0.42)	95.55(±0.33)	89.34(±0.35)	91.50(±0.28)	96.61(±0.67)	98.10(±0.45)

This may be because channels with larger gradients allow shallower networks to train more sufficiently during fine-tuning, leading to better performance under higher pruning-induced errors.

E. Comparisons on Remote Sensing Classification Datasets

To further validate the superiority of the DCCR method, we compared the accuracy and efficiency with mainstream neural networks, including ResNet [1], DenseNet [56], MobileNet [59], [66], RepVGG [19], EfficientNet [57], [58], ShuffleNet [60], [67], Rev-ViT [61], MViTv2 [62], TinyViT [63], RepViT [18], RMT [64], and TransNeXt [65] on three classification datasets (NWPU-RESISC45, UC-Merced Land-Use, and AID). We employed fivefold cross validation for these three datasets, where each dataset was randomly and equally split into five folds. In each fold, one subset was used for training and the other four for validation. This means that each model would be trained five times on each dataset, with 20% of the samples for training in each iteration, and the training samples are nonoverlapping. The final model accuracy was reported as the average and standard deviation over all five iterations, to demonstrate the statistical reliability. During training, all images were randomly scaled, flipped, and cropped to 224 × 224 size. During testing, all images were scaled to 256 × 256 size and center cropped to 224 × 224 size. All methods were trained for 50 epochs on the AID and NWPU datasets and for 30 epochs on the UCM dataset, using the settings specified in Table I. The DCCR method was performed on CSPResNet50, and the pruning ratio was categorized as 0, 0.125, 0.25, and 0.5.

The efficiency and accuracy metrics of these methods were shown in Table VII. CSPResNet50 as the baseline method had the highest accuracy across datasets, but speed and storage are not dominant. Without pruning, the DCCR algorithm reduced the amount of parameters of the baseline model by 37%, the amount of theoretical computation by 35%, and the total number of activations by 53% at the cost of 0.14%–0.56% loss of accuracy. The actual test speed on the GPU was 38% faster and the accuracy was better than other comparison methods. By increasing the pruning ratio, the memory footprint and inference speed of the model can be further improved.

Furthermore, to evaluate the performance of DCCR under different training set ratios, we conducted several sets of experiments on the AID [25], NWPU-RESISC45 [23], and UC-Merced Land-Use [24] datasets. Specifically, for the AID dataset, we randomly utilized 20% and 50% of the data for training; for NWPU, 10% and 20%; and for UCM, 50%

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

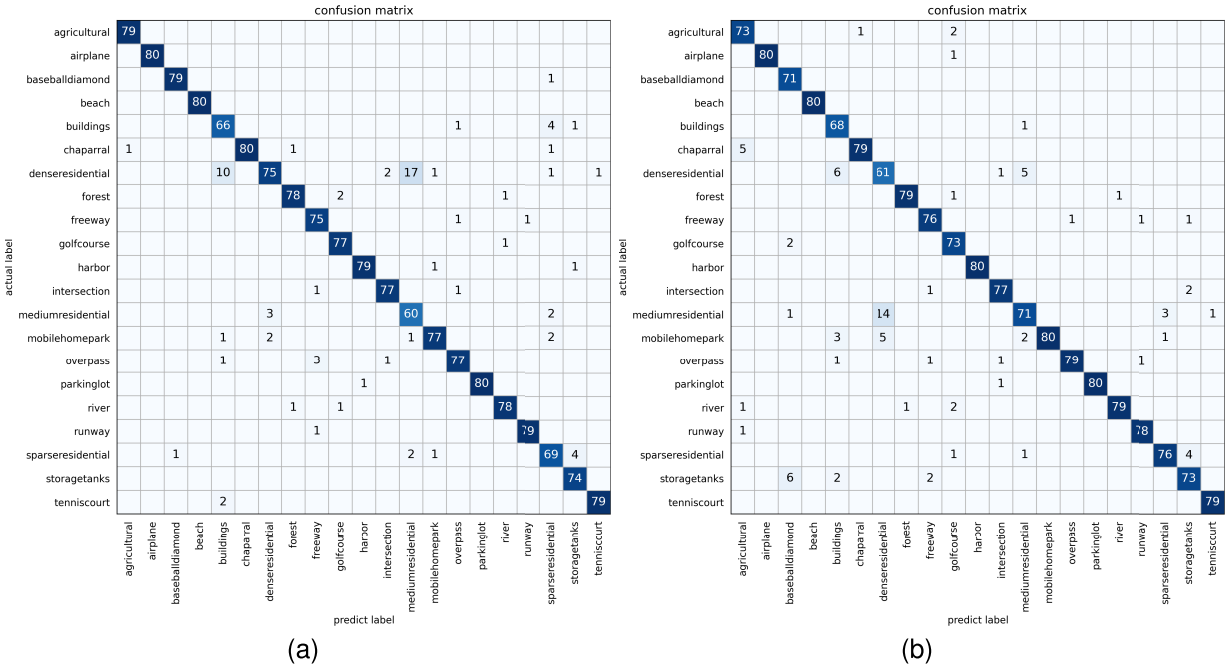


Fig. 10. Confusion matrix of (a) baseline and (b) DCCR-processed network on the UC-Merced Land-Use dataset.

TABLE IX
PERFORMANCE EVALUATION ON FINE-GRAINED CLASSIFICATION DATASET

Neural Network	Accuracy on CUB(%)
CSPResNet50(baseline) [3]	82.48
ResNet50 [1]	78.27
DenseNet121 [56]	79.81
RepVGG-A2 [19]	76.22
EfficientNetV2-b0 [57]	79.79
EfficientNet-b0 [58]	79.62
MobileNetV3-large [59]	76.41
MobileNetV3-small [59]	69.36
ShuffleNetV2 [60]	74.87
Rev-ViT-small [61]	59.60
MViTv2-tiny [62]	65.67
TinyViT-21m [63]	50.18
DCCR($R_{pr} = 0.000$)	**81.39**
DCCR($R_{pr} = 0.125$)	81.19
DCCR($R_{pr} = 0.250$)	80.84
DCCR($R_{pr} = 0.500$)	77.91

TABLE X
PERFORMANCE EVALUATION ON REMOTE SENSING DETECTION DATASET

method	backbone	mAP@50 on DOTA
CADNet [69]	ResNet101	69.90
SCRDet [70]	ResNet101	72.61
RetinaNet [71]	ResNet101	68.72
r3det [72]	ResNet101	71.69
r3det [72]	ResNet152	73.74
s2aNet [27]	ResNet50	74.12
FCOS [68]	ResNet50	71.89
FCOS [68]	CSPResNet50	78.50
FCOS [68]	DCCR($R_{pr} = 0.125$)	76.62

and 80%. Each split ratio was repeated for five independent trials, with all models trained for 100 epochs in each trial. The mean accuracy and standard deviation of the baseline model and DCCR-processed model accuracy were presented in Table VIII. On the AID dataset, DCCR exhibited a minor accuracy loss, ranging from 0.38% to 1.90% with 20% training data and from 0.06% to 0.92% with 50% training data across pruning ratios (0% to 50%). Similar trends were observed on the NWPU and UCM datasets, where accuracy loss remained below 0.7% (pruning ratio under 25%) and 1.9% (pruning ratio under 50%), demonstrating robust performance. These results indicate that the DCCR method performs consistently across different training set ratios, and the accuracy loss tends to decrease as the amount of training data increases. Experimental results and more detailed analyses of other comparative methods are provided in Section II of the supplementary materials.

Fig. 10 demonstrated the confusion matrix for the baseline model and the DCCR-processed model (with a pruning ratio of 0%) on the UCM dataset. These results demonstrated that decomposing the multibranch neural network according to the function and removing the redundant structure stage-by-stage can effectively improve the inference efficiency of the model while ensuring the accuracy. The key to achieving this goal lay in decoupling the network and leveraging the information from the original stage to build a simplified stage, thus alleviating the difficulty of minimizing the reconstruction error via gradient descent method. In this process, SBM was able to convert the hard-to-optimize multibranch structure into an easy-to-optimize single-branch structure and reduced the memory consumption. SLF was able to fuse unnecessary convolutional layers and reduce the number of parameters and the access frequency. SCP was able to improve the efficiency of individual convolutional layers and further compress the number of parameters and reduce the theoretical computation.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
SPEED EVALUATION ON EMBEDDED DEVICES

	RTX3060 Laptop(80W)		JetsonNano(10W)			Ascend 310(8W)
	FP32		FP32	FP16	INT8	FP16
method	latency(ms)	bandwidth(GB/s)	latency(ms)	latency(ms)	latency(ms)	latency(ms)
DenseNet169 [56]	11.12	0.05	85.76	84.63	92.97	6.39
ResNet50 [1]	2.52	0.22	55.01	30.67	54.37	2.44
CSPDarkNet50 [3]	3.35	0.17	66.00	39.20	65.22	3.08
CSPResNet50 [3]	2.20	0.26	51.48	29.33	48.94	2.35
RepVGG-A2 [19]	2.09	0.27	44.07	26.87	46.48	1.86
DCCR($R_{pr} = 0.125$)	1.58	0.36	28.28	17.42	27.30	1.50
DCCR($R_{pr} = 0.250$)	1.36	0.41	25.52	14.87	24.62	1.36
DCCR($R_{pr} = 0.500$)	1.21	0.46	21.11	13.09	20.40	1.26

These three approaches worked together to ultimately achieve a low-loss reconstruction of the multibranch network.

F. Comparisons on Fine-Grained Classification Datasets

Since remote sensing scenarios often involve subtle feature discrimination, we conducted a set of comparative experiments on the fine-grained classification dataset CUB-200-2011 to validate the algorithm's ability to preserve local features. Specifically, we followed the official way to split the training and test sets, maintaining the same optimizer configurations as described in Section IV-B. All models were trained for 100 epochs.

The experimental results in Table IX demonstrate that the DCCR method still delivers robust performance on fine-grained classification datasets, where higher precision in detailed feature extraction is required.

G. Comparisons on Remote Sensing Object Detection Dataset

To further validate the potential of the DCCR method for downstream tasks such as object detection, we conducted experiments on the DOTAv1.0 dataset. The experimental results were shown in Table X. Using a fully convolutional one-stage object (FCOS) detector [68] as the baseline, we replaced its backbone with CSPResNet50 and trained the model for 36 epochs. The optimizer was SGD with a learning rate of 0.0025, a momentum of 0.9, and a weight decay of 0.0001. Data augmentation techniques included random flipping (horizontal, vertical, and diagonal) and multiscale resizing (0.5×, 1×, and 1.5×). The learning rate was reduced by a factor of 10 at the 24th and 33rd epochs. The final model achieved an mAP of 78.50 at an IoU threshold of 50%.

Subsequently, we applied SBM, SLF, and SCP sequentially to the four stages of the backbone network. After optimizing each stage, the model was fine-tuned for ten epochs according to (17). The simplified model achieved an mAP of 76.02 with a reduced parameter size of 78.37 MB. Unlike classification models, the processed backbone in the detection model still required further processing through FPN and FCOSHead to generate the final output. Therefore, we performed an additional 12 epochs of end-to-end training on DOTA, which improved the mAP to 76.62. These results demonstrate that the proposed DCCR algorithm exhibits strong generalization capabilities for downstream tasks like object detection.

H. Benchmarks on Hardware Platforms

To validate the deployment and inference advantages of models optimized by the DCCR method, we conducted speed tests across multiple hardware platforms (RTX3060 Laptop, Jetson Nano, and Ascend 310) using corresponding inference frameworks (TensorRT and CANN) with different precision modes (FP32, FP16, and INT8).

As shown in Table XI, the DCCR-processed CSPResNet network demonstrates superior computational efficiency compared to similarly sized multibranch networks, as it contains only 3 × 3 and 1 × 1 convolutions along with nonlinear layers. This architecture achieves lower processing latency and higher memory bandwidth utilization. Moreover, its simple operator composition ensures excellent compatibility with various post-train quantization tools.

V. CONCLUSION

In this article, we propose a DCCR method for multibranch neural network compression and acceleration, which decomposes the neural network into multiple weakly coupled stages based on the feature map size. Through stage-by-stage reconstruction, we remove the redundant branches, convolutional layers, and channels of the neural network with low accuracy loss. This process does not require end-to-end training or label information from the training set, which allows flexible selection of training data based on specific application scenarios. Finally, an accurate, fast, and memory-friendly single-branch network is obtained. Experiments on public datasets demonstrate that neural network simplification does not depend on end-to-end training and the effectiveness of the DCCR method in improving model inference performance. In the future, we will attempt to extend this method to more complex network structures and other types of data processing tasks, such as infrared, SAR, and hyperspectral data.

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[2] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.

[3] C.-Y. Wang, H.-Y. Mark Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A new backbone that can enhance learning capability of CNN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2020, pp. 1571–1580.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:27:41 UTC from IEEE Xplore. Restrictions apply.

[4] C. Wang, H. M. Liao, and I. Yeh, "Designing network design strategies through gradient path analysis,"*J. Inf. Sci. Eng.*, vol. 39, no. 3, pp. 975–995, 2023. [Online]. Available: http://jise.iis.sinica.edu.tw/JISESearch/pages/View/PaperView.jsf?keyId= 192_2661

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.

[6] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[7] C.-Y. Wang, A. Bochkovskiy, and H.-Y.-M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Vancouver, BC, Canada, Jun. 2023, pp. 7464–7475, doi: 10.1109/CVPR52729.2023.00721.

[8] Q. Xu, Y. Shi, X. Yuan, and X. X. Zhu, "Universal domain adaptation for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4700515.

[9] C. Xu, G. Zhu, and J. Shu, "A lightweight and robust lie group-convolutional neural networks joint representation for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501415.

[10] Q. Zhao, Y. Ma, S. Lyu, and L. Chen, "Embedded self-distillation in compact multibranch ensemble network for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4506415.

[11] T. Zhang, Z. Wang, P. Cheng, G. Xu, and X. Sun, "DCNNet: A distributed convolutional neural network for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5603618.

[12] C. Peng, Y. Li, L. Jiao, and R. Shang, "Efficient convolutional neural architecture search for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 6092–6105, Jul. 2021.

[13] F. Gao et al., "Prototype-based information compensation network for multisource remote sensing data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5513615.

[14] J. Lin, F. Gao, X. Shi, J. Dong, and Q. Du, "SS-MAE: Spatial–spectral masked autoencoder for multisource remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5531614.

[15] F. Gao, X. Jin, X. Zhou, J. Dong, and Q. Du, "MSFMamba: Multiscale feature fusion state space model for multisource remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5504116.

[16] M. Wang, F. Gao, J. Dong, H.-C. Li, and Q. Du, "Nearest neighbor-based contrastive learning for hyperspectral and LiDAR data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5501816.

[17] X. Da, Q. Meng, S. Li, and X. Yuan, "MUDDFormer: Breaking residual bottlenecks in transformers via multiway dynamic dense connections," in *Proc. 42nd Int. Conf. Mach. Learn.*, vol. 267, 2025, pp. 68440–68458. [Online]. Available: https://proceedings.mlr.press/v267/xiao25d.html

[18] A. Wang, H. Chen, Z. Lin, J. Han, and G. Ding, "Rep ViT: Revisiting mobile CNN from ViT perspective," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 15909–15920.

[19] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 13728–13737.

[20] X. Ding, Y. Guo, G. Ding, and J. Han, "ACNet: Strengthening the kernel skeletons for powerful CNN via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1911–1920, doi: 10.1109/ICCV.2019.00200.

[21] J. Wu, D. Zhu, L. Fang, Y. Deng, and Z. Zhong, "Efficient layer compression without pruning," *IEEE Trans. Image Process.*, vol. 32, pp. 4689–4700, 2023.

[22] Y. Fu et al., "DepthShrinker: A new compression paradigm towards boosting real-hardware efficiency of compact neural networks," in *Proc. Int. Conf. Mach. Learn.*, Baltimore, MD, USA, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvári, G. Niu, and S. Sabato, Eds., Jul. 2022, pp. 6849–6862. [Online]. Available: https://proceedings.mlr.press/v162/fu22c.html

[23] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[24] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, San Jose, CA, USA, Nov. 2010, pp. 270–279, doi: 10.1145/1869790.1869829.

[25] G.-S. Xia et al., "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Jul. 2017.

[26] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The caltech-UCSD birds-200–2011 dataset," California Inst. Technol., Pasadena, CA, USA, Tech. Rep. CNS-TR-2011-001, 2011.

[27] J. Han, J. Ding, J. Li, and G.-S. Xia, "Align deep features for oriented object detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5602511.

[28] Y. Long, Y. Gong, Z. Xiao, and Q. Liu, "Accurate object localization in remote sensing images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 5, pp. 2486–2498, May 2017.

[29] C. Lyu et al., "RTMDet: An empirical study of designing real-time object detectors," 2022, *arXiv:2212.07784*.

[30] C. Deng, D. Jing, Z. Ding, and Y. Han, "Sparse channel pruning and assistant distillation for faster aerial object detection," *Remote Sens.*, vol. 14, no. 21, p. 5347, Oct. 2022.

[31] C. Deng, D. Jing, Y. Han, S. Wang, and H. Wang, "FAR-Net: Fast anchor refining for arbitrary-oriented object detection," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[32] T. Liu et al., "Building change detection for VHR remote sensing images via local–global pyramid network and cross-task transfer learning strategy," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4704817.

[33] F. Jiang, M. Gong, H. Zheng, T. Liu, M. Zhang, and J. Liu, "Self-supervised global–local contrastive learning for fine-grained change detection in VHR images," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4400613.

[34] F. Zhang, B. Du, and L. Zhang, "Scene classification via a gradient boosting random convolutional network framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 3, pp. 1793–1802, Mar. 2016.

[35] Y. Liu, Y. Zhong, and Q. Qin, "Scene classification based on multiscale convolutional neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 12, pp. 7109–7121, Dec. 2018.

[36] N. He, L. Fang, S. Li, A. Plaza, and J. Plaza, "Remote sensing scene classification using multilayer stacked covariance pooling," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 12, pp. 6899–6910, Dec. 2018.

[37] J. Fang, Y. Yuan, X. Lu, and Y. Feng, "Robust space–frequency joint representation for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7492–7502, Oct. 2019.

[38] J. Xie, N. He, L. Fang, and A. Plaza, "Scale-free convolutional neural network for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6916–6928, Sep. 2019.

[39] S. Chaib, H. Liu, Y. Gu, and H. Yao, "Deep feature fusion for VHR remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4775–4784, Aug. 2017.

[40] X. Lu, H. Sun, and X. Zheng, "A feature aggregation convolutional neural network for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7894–7906, Oct. 2019.

[41] H. Sun, S. Li, X. Zheng, and X. Lu, "Remote sensing scene classification by gated bidirectional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 82–96, Jan. 2020.

[42] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10558–10578, Dec. 2024.

[43] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2755–2763.

[44] Z. You, K. Yan, J. Ye, M. Ma, and P. Wang, "Gate decorator: Global filter pruning method for accelerating deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst., Annu. Conf. Neural Inf. Process. Syst.*, vol. 32, Vancouver, BC, Canada, H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, Eds., 2019, pp. 2133–2144. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/ b51a15f382ac914391a58850ab343b00-Abstract.html

[45] G. Fang, X. Ma, M. Song, M. Bi Mi, and X. Wang, "DepGraph: Towards any structural pruning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 16091–16101.

[46] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Netw.*, vol. 107, pp. 3–11, Nov. 2018.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.

[47] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," in *Proc. 6th Int. Conf. Learn. Represent.*, Vancouver, BC, Canada, May 2017, pp. 1–16.
[48] Z. Tu, X. Liu, and X. Xiao, "A general dynamic knowledge distillation method for visual analytics," *IEEE Trans. Image Process.*, vol. 31, pp. 6517–6531, 2022.
[49] X. Yang, X. Nan, and B. Song, "D2N4: A discriminative deep nearest neighbor neural network for few-shot space target recognition," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3667–3676, May 2020.
[50] Y. Li and C. Bian, "Few-shot fine-grained ship classification with a foreground-aware feature map reconstruction network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5622812.
[51] G. Gao, P. Zhou, L. Yao, J. Liu, C. Zhang, and D. Duan, "A bi-prototype BDC metric network with lightweight adaptive task attention for few-shot fine-grained ship classification in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5622416, doi: 10.1109/TGRS.2023.3321533.
[52] G.-S. Xia et al., "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 3974–3983.
[53] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 4013–4021, doi: 10.1109/CVPR.2016.435.
[54] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[55] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. 7th Int. Conf. Learn. Represent.*, New Orleans, LA, USA, 2017, pp. 1–16. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7
[56] R. Minetto, M. Pamplona Segundo, and S. Sarkar, "Hydra: An ensemble of convolutional neural networks for geospatial land classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6530–6541, Sep. 2019.
[57] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. 38th Int. Conf. Mach. Learn.*, Jul. 2021, pp. 10096–10106. [Online]. Available: http://proceedings.mlr.press/v139/tan21a.html
[58] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Mach. Learn.*, Long Beach, CA, USA, Jun. 2019, pp. 6105–6114. [Online]. Available: http://proceedings.mlr.press/v97/tan19a.html
[59] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.
[60] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proc. Eur. Conf. Comput. Vis.*, in Lecture Notes in Computer Science, Munich, Germany, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds., Cham, Switzerland: Springer, 2018, pp. 122–138, doi: 10.1007/978-3-030-01264-9.
[61] K. Mangalam et al., "Reversible vision transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, Jun. 2022, pp. 10820–10830, doi: 10.1109/CVPR52688.2022.01056.
[62] Y. Li et al., "MViTv2: Improved multiscale vision transformers for classification and detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, Jun. 2022, pp. 4794–4804, doi: 10.1109/CVPR52688.2022.00476.
[63] K. Wu et al., "TinyViT: Fast pretraining distillation for small vision transformers," in *Proc. 17th Eur. Conf. Comput. Vis.*, Tel Aviv, Israel, S. Avidan, G. J. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds., Cham, Switzerland: Springer, Oct. 2022, pp. 68–85, doi: 10.1007/978-3-031-19803-8_5.
[64] Q. Fan, H. Huang, M. Chen, H. Liu, and R. He, "RMT: Retentive networks meet vision transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 5641–5651.
[65] D. Shi, "TransNeXt: Robust foveal visual perception for vision transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 17773–17783.
[66] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.
[67] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 6848–6856.
[68] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9626–9635, doi: 10.1109/ICCV.2019.00972.
[69] G. Zhang, S. Lu, and W. Zhang, "CAD-Net: A context-aware detection network for objects in remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 10015–10024, Dec. 2019.
[70] X. Yang et al., "SCRDet: Towards more robust detection for small, cluttered and rotated objects," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Jun. 2019, pp. 8231–8240.
[71] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020, doi: 10.1109/TPAMI.2018.2858826.
[72] X. Yang, J. Yan, Z. Feng, and T. He, "R3Det: Refined single-stage detector with feature refinement for rotating object," in *Proc. 35th AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 3163–3171, doi: 10.1609/aaai.v35i4.16426.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 13:47:41 UTC from IEEE Xplore. Restrictions apply.